OPERATING SYSTEMS AND SYSTEMS PROGRAMMING

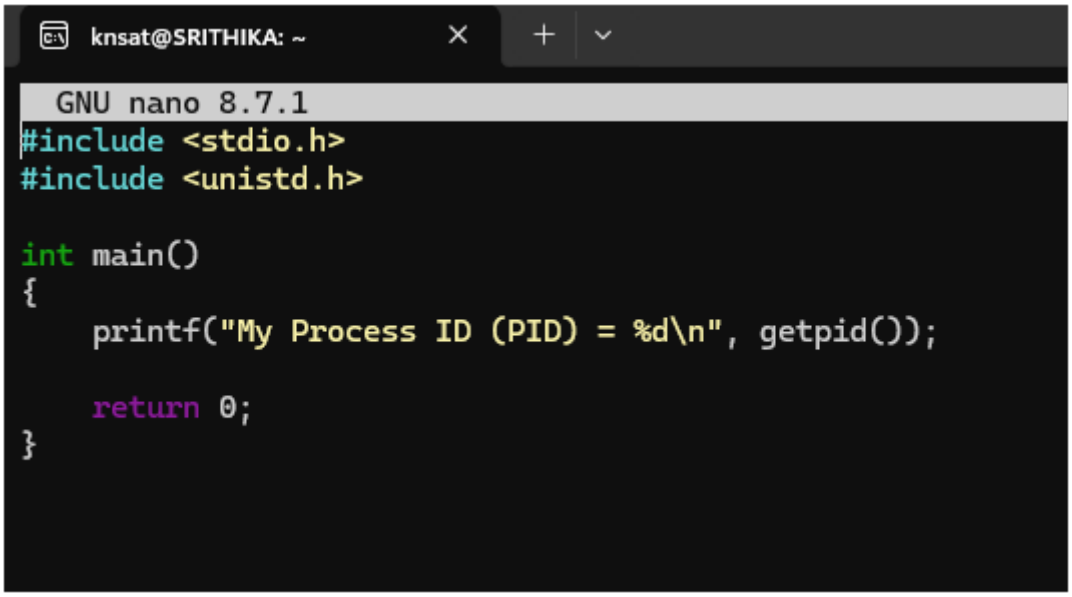
Sec:- 2

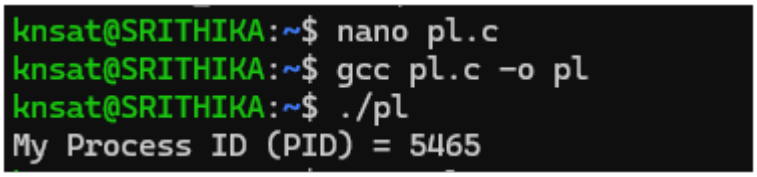
Name:- Srithika Kambhammettu

RollNo:- 2520030361

OS Practical Input and Output

1. processid





2. creating child using fork()

knsat@SRITHIKA: ~

GNU nano 8.7.1

```c
#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t pid = fork();

    if(pid == 0)
    {
        printf("Child Process\n");
        printf("PID = %d\n", getpid());
    }
    else
    {
        printf("Parent Process\n");
        printf("PID = %d\n", getpid());
    }

    return 0;
}
```

```
knsat@SRITHIKA:~$ nano p2.c
knsat@SRITHIKA:~$ gcc p2.c -o p2
knsat@SRITHIKA:~$ ./p2
Parent Process
PID = 5511
Child Process
PID = 5512
```

3. Parent Waits for Child (wait())

knsat@SRITHIKA: ~

GNU nano 8.7.1

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid = fork();

    if(pid == 0)
    {
        printf("Child Process Executing...\n");
        sleep(3);
        printf("Child Finished\n");
    }
    else
    {
        wait(NULL);
        printf("Parent Process Continues\n");
    }

    return 0;
}
```

```
knsat@SRITHIKA:~$ nano p3.c
knsat@SRITHIKA:~$ gcc p3.c -o p3
knsat@SRITHIKA:~$ ./p3
Child Process Executing...
Child Finished
Parent Process Continues
```

4. Multiple Child Processes

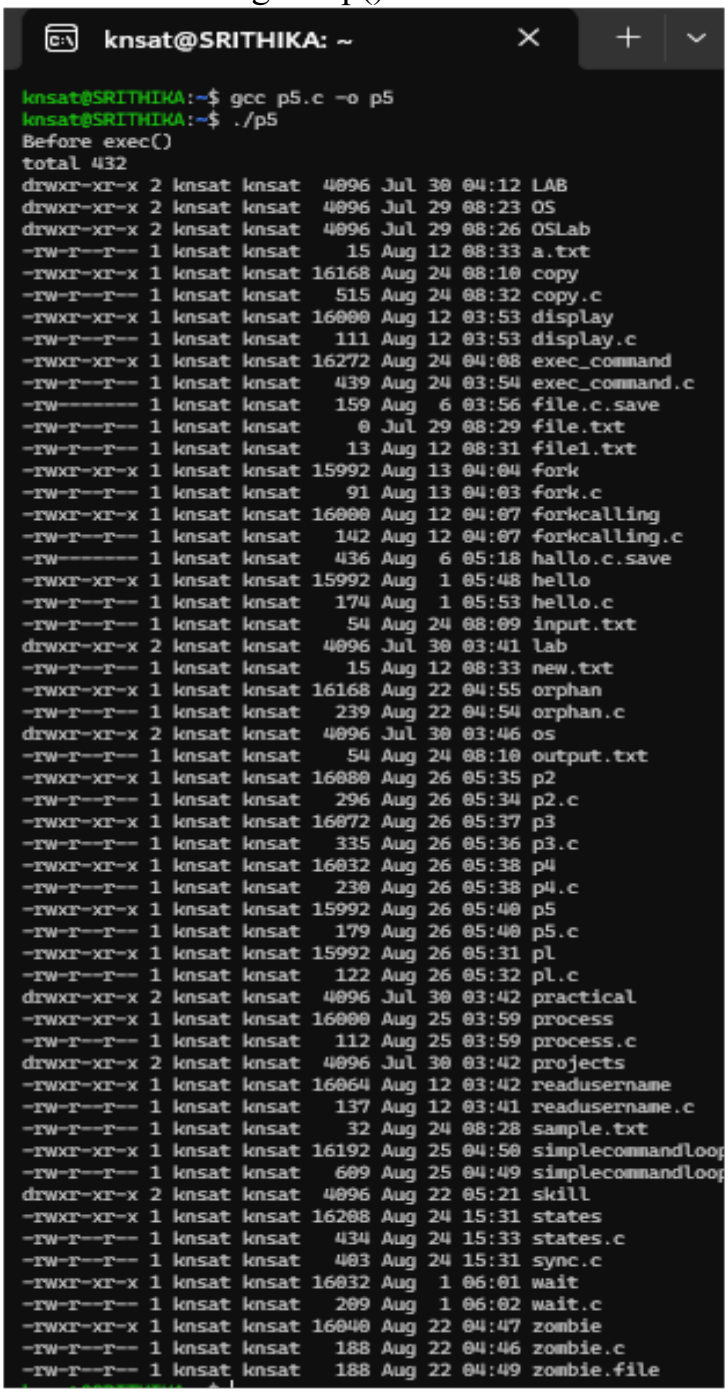
knsat@SRITHIKA: ~

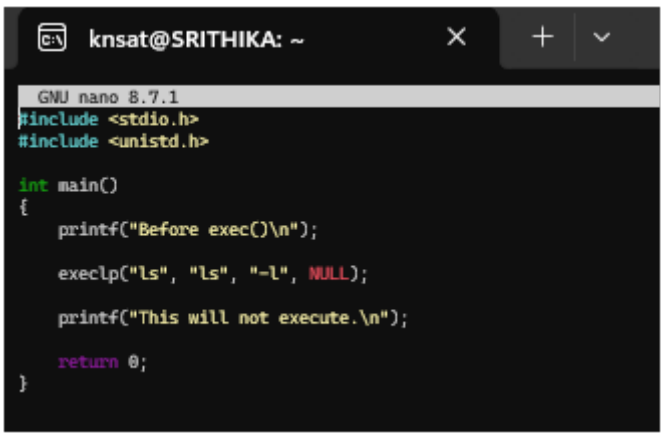
GNU nano 8.7.1

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    for(int i=1; i<=3; i++)
    {
        if(fork()==0)
        {
            printf("Child %d PID = %d\n", i, getpid());
            return 0;
        }
    }

    return 0;
}
```

```
knsat@SRITHIKA:~$ nano p4.c
knsat@SRITHIKA:~$ gcc p4.c -o p4
knsat@SRITHIKA:~$ ./p4
Child 1 PID = 5614
Child 2 PID = 5615
Child 3 PID = 5616
```

5. Execute linux command using exclp()





6. Parent and Child Executing Concurrently

knsat@SRITHIKA: ~

```
 GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>

int main()
{
    fork();

    for(int i=1; i<=5; i++)
    {
        printf("PID: %d  Count: %d\n", getpid(), i);
        sleep(1);
    }

    return 0;
}
```

```
knsat@SRITHIKA:~$ nano p6.c
knsat@SRITHIKA:~$ gcc p6.c -o p6
knsat@SRITHIKA:~$ ./p6
PID: 5707  Count: 1
PID: 5708  Count: 1
PID: 5707  Count: 2
PID: 5708  Count: 2
PID: 5707  Count: 3
PID: 5708  Count: 3
PID: 5707  Count: 4
PID: 5708  Count: 4
PID: 5707  Count: 5
PID: 5708  Count: 5
knsat@SRITHIKA:~$
```

7. Zombie Process

```c
#include <stdio.h>
#include <unistd.h>
int main()
{
pid_t pid = fork();
if(pid == 0)
{
printf("Child Existing\n");
return 0;
}
else
{
sleep(15);
printf("Parent Finished\n");
}
return 0;
}
```

```
knsat@SRITHIKA:~$  nano zombie.file
knsat@SRITHIKA:~$ gcc zombie.c -o zombie
knsat@SRITHIKA:~$ ./zombie
Child Existing
Parent Finished
```

8. Orphan Process

knsat@SRITHIKA: ~

GNU nano 8.7.1

```c
#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t pid = fork();

    if(pid > 0)
    {
        printf("Parent Exiting\n");
        return 0;
    }
    else
    {
        sleep(5);
        printf("Child PID : %d\n", getpid());
        printf("New Parent PID : %d\n", getppid());
    }

    return 0;
}
```

```
knsat@SRITHIKA:~$ nano p7.c
knsat@SRITHIKA:~$ gcc p7.c -o p7
knsat@SRITHIKA:~$ ./p7
Parent Exiting
knsat@SRITHIKA:~$ Child PID : 5754
New Parent PID : 738
```